

Manual técnico — GrillaA (web/)

Stack

La aplicación web de GrillaA está construida sobre un stack moderno, ligero y orientado a rendimiento en dispositivos Android de gama media:

- **Next.js 16** con el App Router y **Turbopack** como bundler de desarrollo.
- **React 19** con Server Components por defecto.
- **TypeScript** en todo el código.
- **Tailwind CSS 4** para estilos utilitarios.
- **Plus Jakarta Sans** como tipografía principal (cargada vía `next/font`).
- **shadcn/ui** (base radix) para las primitivas accesibles: Sheet, Dialog, DropdownMenu, AlertDialog, Accordion.
- **lucide-react** para iconografía consistente y accesible.

No hay librerías de estado global ni frameworks CSS-in-JS. La superficie de dependencias se mantiene deliberadamente pequeña para facilitar el mantenimiento por parte de un equipo académico.

Estructura del proyecto

El código vive dentro de la carpeta `web/`:

- `web/src/app/` — Rutas del App Router de Next.js. Cada carpeta es una ruta (`/`, `/tutorial`, `/catalogo`, `/como-armar`, `/wizard`, `/chat`, `/proyectos`, `/contacto`, `/metodologia`, `/proyecto`, `/reset`). Incluye `layout.tsx` raíz con el banner y footer demo, y `page.tsx` por ruta.
- `web/src/components/` — Componentes reutilizables (UI primitives, cards de proyecto, pasos del wizard, burbujas de chat, banner demo, footer demo, etc.).
- `web/src/lib/` — Lógica pura y datos: `animals.ts` (catálogo de animales y etapas con metas de proteína), `chat-knowledge.ts` (base de conocimiento del chat informativo), `projects-store.ts` (hook de consultas guardadas), `journey.ts` y `use-paso.ts` (puntero del recorrido), `content/*.ts` (contenido de tutorial, catálogo, guía cajas, metodología).

Cómo correr en local

Requisitos: **Node.js 22 o superior** y **pnpm**.

```
cd web
pnpm install
pnpm dev
```

La app queda disponible en `http://localhost:3000`. Turbopack se activa automáticamente en `pnpm dev`. Para build de producción: `pnpm build && pnpm start`.

Variables de entorno

Hoy la app **no requiere ninguna variable de entorno obligatoria** para funcionar: toda la persistencia es local. Las siguientes variables están reservadas para cuando se active la autenticación en la rama preparada:

- `DATABASE_URL` — URL de conexión a Postgres (Railway).
- `NEXTAUTH_URL` — URL pública de la app.
- `NEXTAUTH_SECRET` — Secreto para firmar sesiones.
- `GOOGLE_CLIENT_ID` — Client ID de OAuth de Google.
- `GOOGLE_CLIENT_SECRET` — Client secret de OAuth de Google.
- `NEXT_PUBLIC_ENABLE_AUTH` — Feature flag (`"true"` o `"false"`) que activa el flujo de login y la persistencia remota.

Se deben definir en `web/.env.local` para desarrollo y en el panel de Vercel para producción.

Despliegue en Vercel

El despliegue se hace por integración directa con GitHub:

1. Importar el repositorio en Vercel.
2. Configurar **Root Directory** = `web`.
3. **Framework Preset** = `Next.js` (se detecta automáticamente).
4. Dejar los comandos por defecto (`pnpm install`, `pnpm build`).
5. Definir variables de entorno solo cuando se active Auth.

Cada push a `main` despliega producción; los PRs generan preview deployments automáticos.

Persistencia

En la versión actual toda la información del usuario (proyectos, historial de chat, preferencias) se guarda en **localStorage** del navegador. Esto simplifica el MVP y evita costos de infraestructura mientras el modelo se entrena.

Existe una rama preparada con integración de **NextAuth.js** (proveedor Google) y **Railway Postgres** como base de datos, activable mediante el feature flag `NEXT_PUBLIC_ENABLE_AUTH`.

Cuando el flag es `true`, los hooks de persistencia cambian su fuente de datos sin que las páginas necesiten modificarse.

Cómo agregar contenido

La mayoría del contenido es data estática en TypeScript, versionada en git:

- **Animales y etapas:** editar `web/src/lib/animals.ts` . Cada entrada define nombre, etapas y metas internas de proteína.
- **Dietas en estudio:** editar `web/src/lib/animals.ts` (constante `DIETS`). Actualmente D1 harina de bore, D2 harina de botón de oro, D3 salvado de trigo, todas con misma base (choclo 10% + avena 10%).
- **Base de conocimiento del chat:** editar `web/src/lib/chat-knowledge.ts` con preguntas frecuentes y respuestas curadas.
- **Contenido del tutorial:** editar `web/src/lib/content/tutorial.ts` .
- **Catálogo de grillos:** editar `web/src/lib/content/catalog.ts` .
- **Guía de armado:** editar `web/src/lib/content/enclosure.ts` .
- **Metodología y referencias:** editar `web/src/lib/content/methodology.ts` .

Cualquier cambio en estos archivos se refleja de inmediato en la UI sin migraciones.

Convenciones

- **Tamaño de fuente base: 18px**, pensado para lectura cómoda en celulares de audiencia adulta rural.
- **Contraste alto** en todos los textos (mínimo WCAG AA).
- El **banner** superior es obligatorio en todas las rutas: recuerda al usuario que está en una versión en pruebas. Se monta en el `layout.tsx` raíz.
- **Server Components por defecto.** Solo se marca `"use client"` cuando el componente necesita estado local, efectos o handlers de eventos (wizard, chat, formularios).
- Nombres de archivos y componentes en inglés; textos visibles en español, tratando de "usted".
- Nunca prometer resultados ni dar fechas específicas en copy: usar "le sugerimos", "meta interna", "dietas en estudio", "condiciones objetivo".
- **Un botón primario y un enlace secundario por pantalla como máximo.** Use `components/step-footer.tsx` , que impone la regla.
- El pie del sitio no se renderiza en `/chat` ni en `/reset` : son pantallas de alto completo y el pie les rompe la distribución. La lógica vive en `components/site-footer.tsx` .
- **No registrar service workers.** El `public/sw.js` es un kill-switch permanente sin `fetch` handler; ver el historial en ese archivo antes de tocarlo.

Migración futura a Railway/NextAuth

La migración está diseñada para ser transparente para las páginas, mediante un **feature flag** (`NEXT_PUBLIC_ENABLE_AUTH`) y dos hooks abstractos:

- **useProjects** — Devuelve los proyectos del usuario. Su implementación interna elige entre lectura/escritura en `localStorage` (modo local) o llamadas a la API con la sesión de NextAuth (modo remoto).
- **useChat** — Igual, para los hilos de conversación del chat: local hoy, remoto cuando el flag está activo.

Esto permite activar la nube de forma incremental sin reescribir componentes.

Accesibilidad

- Todos los botones e iconos interactivos tienen **aria-label** descriptivo en español.
- El estado **focus-visible** está estilizado explícitamente en Tailwind para navegación por teclado.
- Se respeta **prefers-reduced-motion**: las animaciones del wizard y de las transiciones se desactivan cuando el usuario lo indica en su sistema.
- Contraste de color validado en modo claro; jerarquía tipográfica clara con Plus Jakarta Sans.
- Tap targets mínimos de 44x44 px en toda la UI móvil.